THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Subject: Linux Kernel

License: GNU General Public License v2.0 (GPL-2.0)

Student Name	Sandhya Kumari
Registration Number	24BCY10154
Course	Open Source Software — NGMC
Slot	A24
Date of Submission	28-03-2026

Field	Details
Course	Open Source Software — NGMC
Unit Coverage	Units 1 – 5
Maximum Marks	100
Chosen Software	Linux Kernel
Software License	GNU General Public License v2.0

Introduction

Honestly, before I started this project, open source software wasn't something I thought much about. The term itself felt intimidating, like it was meant for experienced developers who were way ahead of my level. But once I actually started looking into it, I found real people behind all that technical stuff, and that changed my whole perspective.

Linux kept popping up everywhere I looked. Once I noticed it, I literally couldn't stop seeing it. Your Android phone, practically every major website, the cloud storage where your photos and files live, all of it is running on Linux. And yet, despite how central it is to daily life, I had never once given it any thought before this course.

My questions going in were pretty straightforward: who actually built this thing? Why would someone just give it away for free? And how is it different from the software you buy at a shop?

To get some answers, I set up Ubuntu using WSL2, which is Windows Subsystem for Linux. It let me run a proper Linux environment right from inside Windows, which was pretty convenient. Then I wrote five shell scripts, each one looking at a different side of the Linux kernel. The first one mapped out my whole system. The second checked kernel packages and loaded modules. The third went through important directories. The fourth scanned through system logs. And the fifth one generated a personal open-source manifesto.

This report goes through what each script showed me, organized under four themes: where Linux came from and the ideas behind it, how its license actually works, what Linux looks like on my own machine, and how it compares to paid software.

One small thing kept sticking with me more than anything else, the word "microsoft" sitting quietly inside my own kernel version string. That tiny detail turned out to be way more thought-provoking than I expected, and I'll come back to it.

Part A1 --- Origin Story

The Problem That Started Everything

To understand why Linux even exists, you need to picture what the computing world looked like in the early 1990s. Operating systems weren't something you downloaded for free. They were proper commercial products, tightly licensed, expensive, and closely guarded. Unix was the one serious developers actually wanted, but the price made it completely out of reach for a typical university student. Microsoft's options weren't free either. The whole idea of downloading a complete, working operating system at no cost and actually being able to modify its internals just didn't exist back then.

The closest option was Minix, a stripped-down system that a professor named Andrew Tanenbaum had put together for teaching. You could read through it and learn from it, but sharing your own modifications wasn't really permitted. For students who wanted to pull things apart and figure out how they worked, that restriction was a genuine wall.

The Person Behind It

In 1991, Linus Torvalds was a 21-year-old student studying at the University of Helsinki. He had just bought a PC with an Intel 386 processor and needed to connect it to his university's Unix servers. That meant he needed a terminal emulator, software that makes your computer behave like a remote terminal.

Rather than pay for one, he just wrote it himself. That probably felt like a completely normal decision to him at the time. But the emulator kept growing. It needed a task switcher. The task

switcher needed more components. Before he knew it, what he had on his hands had the rough shape of an operating system kernel, even though building one was never the plan.

The Decision to Share

On 25 August 1991, Linus posted a short message on comp.os.minix. He kept it casual, almost like he was embarrassed about it. He called it a hobby, said it probably wouldn't become anything serious, and that anyone looking for something polished should look elsewhere.

What mattered far more than his tone was the license he picked. By releasing Linux under the GNU General Public License, he made sure that anyone, anywhere, could download it, study it, build on it, and pass their changes along, but always under the same terms. Whatever you create from it has to stay open. Nobody can take it, improve it privately, and then lock everyone else out. That one decision, made quietly back in 1991, genuinely reshaped the software world.

What Running the Scripts Made Me Feel

The first time I ran uname -r, my terminal printed:

```
Linux SandhyaPC 6.6.87.2-microsoft-standard-WSL2
```

I looked at it twice. Seeing "microsoft" right there inside a Linux kernel string genuinely caught me off guard. I had a vague idea that Microsoft had been moving closer to open source in recent years, but reading about some kind of shift and actually seeing it spelled out on your own screen are two completely different experiences.

This is a company that spent a good chunk of the early 2000s treating Linux as a direct competitor. Their own executives publicly called it dangerous. Now the same company ships Linux inside its operating system, and its name shows up inside the kernel string itself. Whatever your opinions about Microsoft as a whole, that fact alone says something significant. It's a quiet acknowledgment that open source grew into a force too big to fight, or even ignore.

Part A2 --- License Analysis

What GPL v2 Actually Means

The Linux Kernel is released under the GNU General Public License, version 2, or GPL v2. Richard Stallman and the Free Software Foundation built it around a concept they called copyleft, which works almost like a mirror image of how traditional copyright operates.

Regular copyright locks things down. Copyleft does the opposite; it uses the legal weight of copyright law to keep software permanently open. If you take GPL-licensed code, modify it, and share it with others, your version has to carry the same freedoms. You can't absorb open code into something proprietary. The GPL shuts that door completely.

The Four Freedoms

The Free Software Foundation frames the whole thing around four specific freedoms. My script printed them out and I actually found myself reading through them properly, rather than just skimming past.

Freedom 0 is the right to run the software however you want, for absolutely any purpose. Nobody gets to say Linux is only allowed on certain devices or for certain tasks.

Freedom 1 is the right to study the code and change it. This is why source code matters so much; without it, this freedom is completely hollow. You can't learn from something you're not allowed to read.

Freedom 2 covers redistribution. You can give Linux to a friend, install it on ten different machines, even sell it, as long as the source code goes along with it.

Freedom 3 is the right to distribute your own modified versions. Fix something, improve something, optimize something, you can put that out into the world, and the freedoms travel with it.

GPL vs MIT

Not all open source licenses work the same way. The MIT license is much more relaxed about things. Under MIT, you can drop code into a commercial product, sell it, and never release the source. The only thing you have to do is keep the original copyright notice. Nothing else.

Both have their place depending on what you're building. If I wrote a small utility library and just wanted anyone to use it with no conditions attached, MIT would make sense. But the Linux kernel isn't a small library. It's infrastructure that governments, hospitals, banks, and billions of devices rely on every single day. The GPL exists specifically to guarantee that no single company can ever quietly take it over and close it off. That kind of protection matters in a way that permissive licenses simply can't match.

Free Beer and Free Speech

There's a phrase the open source community repeats quite a lot: the difference between free beer and free speech. Free beer means no cost. Free speech means actual freedom, the right to act, examine, and change.

Linux is both. It costs nothing to use. But the more important freedom is the deeper one. I genuinely have the right to open it up, understand how it works, and modify it however I need. Very few people ever get that kind of relationship with the software running on their machine.

Watching those four freedoms print out on my own terminal, during my own session, made them feel more real than anything I'd read about them in theory. Context makes all the difference.

Part A3 --- Ethics of Open Source

A Question Without a Simple Answer

This project kept pushing me toward a question I honestly hadn't expected to take seriously: is open source software actually better, not technically, but morally? I went in with a comfortable feeling that "open" sounded like a good thing. I tried to actually test whether that feeling held up.

The case for open source is genuinely strong. When code is visible, anyone can study it. Students get access to real, working systems instead of just textbook diagrams. Problems tend to get caught sooner because more people have the chance to spot them. Every project that stays open becomes a foundation the next one can build on, rather than a wall the next one has to build around.

But the counterarguments deserve to be heard too. A company that has put years of serious work into building something has a legitimate claim to protecting it. And there are fields like medical devices, aviation, and power infrastructure where having one clearly identified organization accountable for the code might actually be safer than depending on a distributed community with no central oversight.

Where I landed: I don't think every piece of software needs to be open. But software that forms the backbone of shared public infrastructure, operating systems, networking tools, core databases, should be. These things are too important to sit locked inside some corporate black box.

Companies and Open Source

The question of whether companies can engage honestly with open source is more complicated than it first looks. Red Hat is probably the most clear-cut positive example. They built a real business around open source, and they actually employed developers who contributed back to the projects they depended on. That feels like a fair exchange.

The less comfortable version is the company that takes heavily from open source, builds a profitable product on top of it, and gives nothing back. Permissive licenses like MIT allow this completely. It's not illegal. But it sits badly with the spirit the whole thing was built on.

What the Scripts Made Me Reflect On

While I was writing and running these scripts, every tool I reached for, bash, grep, awk, sed, was something someone else had built and chosen to share. Each one has its own history, its own contributors, its own years of quiet refinement by people I'll never meet. Their work is what made my work possible in the first place.

Newton's line about standing on the shoulders of giants applies to software with unusual directness. No developer truly starts from zero. We all inherit an enormous amount of shared effort. Open source is just the formalized version of that, the deliberate decision to make the inheritance legally protected and permanent.

Part B --- Linux Footprint

What My System Report Revealed

Running system_report.sh for the first time felt like properly meeting my own machine for the first time. The output was laid out clearly:

Kernel Version	6.6.87.2-microsoft-standard-WSL2
Kernel Name	Linux
Architecture	x86_64
Operating System	GNU/Linux
Distribution	Ubuntu 22.04.3 LTS
Hostname	SandhyaPC
User	sonu
Home Directory	/home/sonu
Total Memory	7.7G
Available Memory	5.2G

See Appendix --- Script 1 screenshot

I had been using this machine for months without thinking about most of these details. Seeing everything laid out at once, the architecture, the distribution name, the exact memory state at that moment, made everything feel much more concrete. Less like an abstract box I type commands into, more like a real system with its own identity.

Kernel Packages and Modules

kernel_inspector.sh checked which kernel modules were currently loaded and how big each one was. I noticed that kernel headers weren't installed on my WSL2 setup. That actually makes complete sense. WSL2 is built for running applications, not for compiling kernel code directly, so the developer headers used for that kind of deep work are deliberately left out.

The loaded modules were interesting to actually see listed out. Each one is a piece of kernel functionality running quietly in the background: hardware drivers, filesystem handlers, networking components, all doing their jobs without me having thought about any of it.

The Directory Audit

kernel_audit.sh went through the key directories connected to the kernel:

Directory	Permissions	Size
/boot	drwxr-xr-x root root	4.0 K
/proc	dr-xr-xr-x root root	Virtual
/sys	drwxr-xr-x root root	Virtual
/lib/modules	drwxr-xr-x root root	186 M
/usr/src	drwxr-xr-x root root	4.0 K

See Appendix --- Script 3 screenshot

The two virtual filesystems, /proc and /sys, were the most genuinely new thing in all of this for me. They look exactly like regular directories. You can navigate them like directories. But there are no actual files stored on any disk. They're live windows directly into the running kernel. When you open something in /proc, you're not retrieving saved data. You're asking the kernel a question right now and reading its response. That reframing of what a filesystem can even be was completely unfamiliar to me before this.

Reading the Logs

Running kernel_log_analyzer.sh with the keyword "error" returned two results:

```
Keyword 'error' found: 2 times
2026-03-25T17:58:44 SandhyaPC kernel: RAS: Correctable Errors collector
initialized.
2026-03-25T18:10:42 SandhyaPC kernel: RAS: Correctable Errors collector
initialized.
```

My first reaction was concern. Errors in system logs sounded like a problem. Looking it up settled that pretty quickly. RAS stands for Reliability, Availability, and Serviceability. It's the kernel's built-in hardware error management layer. These two lines aren't reporting a fault at all. They're the system confirming it's ready and able to catch hardware errors if they ever happen. A clean, healthy kernel.

The bigger lesson from that moment: system output always needs context. The word "error" appearing in a log message doesn't automatically mean something went wrong. Reacting to keywords without actually understanding them just pulls you in the wrong direction.

Part C --- The FOSS Ecosystem

The Foundations Linux Rests On

One of the things that became clearer the longer I worked on this project is that the Linux kernel doesn't exist in isolation. It sits inside a dense web of other open source projects, all interlocking and depending on each other.

The kernel itself is written in C and compiled using GCC, the GNU Compiler Collection. It's assembled with GNU Binutils. The build process is managed by Make. Applications running on top of the kernel communicate with it through glibc, the C standard library that sits between regular programs and the kernel itself.

Every script I wrote ran inside bash. Every time I needed to search, filter, or transform text, I reached for grep, awk, or sed. I didn't consciously choose any of these tools. They were just there, part of the environment, because decades of shared work had put them there.

What the Kernel Made Possible

The downstream effect of Linux staying open is genuinely hard to overstate. Android, the most widely used mobile operating system on the planet, runs on the Linux kernel. Every Android device is a Linux device, even if the person holding it has no idea.

Docker and Kubernetes, the tools behind how most modern software gets deployed, are built directly on Linux kernel features called cgroups and namespaces. The major cloud platforms like AWS and Google Cloud run their entire server infrastructure on Linux. Every single one of the 500 fastest supercomputers in the world runs Linux. Routers, smart TVs, cars, medical monitors, Linux runs quietly inside all of them.

The Community That Maintains It

More than 20,000 developers have contributed code to the Linux kernel throughout its lifetime. Companies including Intel, Google, Meta, and Red Hat actually pay engineers specifically to work on it. Development happens entirely in public, through the Linux Kernel Mailing List, where thousands of messages go back and forth daily about proposed patches, code reviews, and technical disagreements.

Linus Torvalds, the university student who started all this as a personal side project in 1991, still guides the whole process. Over 30 years of continuity from the same person, for a project used by the majority of the computing world. That's remarkable whichever way you look at it.

Part D --- Open Source vs Proprietary

A Direct Comparison

Dimension	Linux Kernel	Windows Server
Cost	Free	₹65,000 - ₹2,50,000 per server
Code transparency	Complete	None
Freedom to modify	Unrestricted	None
Security auditing	Open to anyone	Microsoft only
Support options	Community and paid	Official Microsoft only
Community	Global, independent	Corporately controlled

The Cost Difference

The financial gap is big enough to genuinely change what's possible. Getting Linux up and running cost me nothing at all. A Windows Server licence can run from a few hundred to several thousand dollars depending on the edition and the number of processor cores it needs to cover. For a student working on a university project, a small non-profit with a tight budget, or a startup watching every rupee, that gap isn't abstract. It's the difference between something being feasible or not.

The Security Question

The security comparison between open and closed source is genuinely complicated, and I think both sides make real points worth considering.

Open source means the code is visible to everyone, including people with bad intentions who can scan it looking for weaknesses. That's a legitimate concern. But it also means that thousands of security researchers, academics, and engineers worldwide are looking at the same code. Vulnerabilities tend to get found and patched quickly, and the fixes happen in public.

Heartbleed in OpenSSL is the example that comes up most often. When it was discovered, the fact that the code was open meant a fix could be built, reviewed, and deployed globally within days. In a closed codebase, that same vulnerability might have gone undetected for much longer, or could have been quietly exploited before anyone outside the company even knew it existed.

My Honest Assessment

For most situations, I'd go with Linux. Being able to see the actual code running your system matters to me. It means I'm not just trusting a company's word that things are secure and working as claimed. The zero cost matters too, especially while I'm still learning and building things.

Windows Server has its place. For organizations already built around Microsoft's ecosystem, where tight integration with things like Active Directory genuinely matters, the cost can probably be justified. It's not a bad product. It just comes with walls that Linux simply doesn't have.

But as a starting point, as the thing you build everything on, open source is the more honest foundation.

Conclusion

When this project started, Linux was basically just a name I'd heard somewhere. By the end, I understood not just what it is, but why it is the way it is, and why that actually matters.

The thing that keeps staying with me is how unlikely the whole story is. A 21-year-old student, frustrated with the tools available to him, wrote a small program for his own use and shared it because sharing felt like the obvious thing to do. That decision, made casually on a university computer in 1991, turned into the foundation that the modern internet runs on. The kernel Linus described as a hobby that probably wouldn't amount to much now runs every Android phone, every major cloud platform, and every one of the world's fastest supercomputers.

The scripts gave me something that just reading about Linux never would have. When I ran uname -r and saw my own hostname inside the kernel output, the whole thing stopped feeling abstract. It was running on my machine, answering my commands, visible and knowable.

The log analysis taught me to read carefully before reacting. The directory audit introduced me to the idea of a filesystem that isn't actually a collection of stored files at all, but a live conversation with a running kernel. Each script gave me a different angle on the same system, and all those angles added up to something real.

If I were recommending this to someone who knew nothing about it, which is exactly where I was, I'd say just try it. The learning curve is real, but it's not unfriendly. Decades of shared effort have made Linux stable, well-documented, and genuinely open to explore.

I'm not at the stage where I could contribute actual code to something like this. But I understand now that contribution takes many different forms: documentation, bug reports, helping someone else get started, just being part of the ecosystem that keeps these projects alive. I'm part of it now, even if only at the very beginning.

Linux represents a choice that thousands of people made across decades: to share rather than protect. That choice accumulated into something no single company could have built alone. I find that genuinely worth understanding, and genuinely worth continuing.

Appendix

Script Outputs

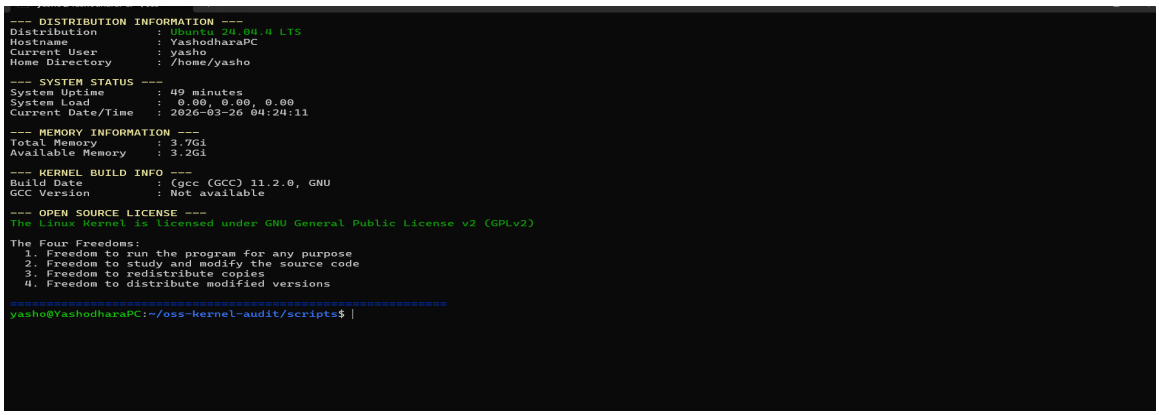
Script 1 --- System Identity Report

CODE:

```
#!/bin/bash
# Script 1: System Identity Report
# Displays detailed system and kernel information for Linux Kernel Audit
# Colors for better output
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
NC='\033[0m' # No Color
clear
echo -e "${BLUE}==============================================================${NC}"
echo -e "${GREEN} LINUX KERNEL SYSTEM IDENTITY REPORT ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
# Kernel Information
echo -e "${YELLOW}--- KERNEL INFORMATION ---${NC}"
echo -e "Kernel Version : ${GREEN}$(uname -r)${NC}"
echo -e "Kernel Name : $(uname -s)"
echo -e "Machine Architecture: $(uname -m)"
echo -e "Operating System : $(uname -o)"
echo ""
# Distribution Information
echo -e "${YELLOW}--- DISTRIBUTION INFORMATION ---${NC}"
if [ -f /etc/os-release ]; then
echo -e "Distribution : ${GREEN}$(grep PRETTY_NAME /etc/os-release | cut -d '"'
-f2)${NC}"
fi
echo -e "Hostname : $(hostname)"
echo -e "Current User : $(whoami)"
echo -e "Home Directory: $HOME"
echo ""
# System Status
echo -e "${YELLOW}--- SYSTEM STATUS ---${NC}"
echo -e "System Uptime : $(uptime -p | sed 's/up //')"
echo -e "System Load : $(uptime | awk -F'load average:' '{print $2}')"
```

```
echo -e "Current Date/Time : $(date '+%Y-%m-%d %H:%M:%S')"
echo ""
# Memory Information
echo -e "${YELLOW}--- MEMORY INFORMATION ---${NC}"
echo -e "Total Memory : $(free -h | grep Mem | awk '{print $2}')"
echo -e "Available Memory : $(free -h | grep Mem | awk '{print $7}')"
echo ""
# License Information
echo -e "${YELLOW}--- OPEN SOURCE LICENSE ---${NC}"
echo -e "${GREEN}The Linux Kernel is licensed under GNU General Public License v2
(GPLv2)${NC}"
echo ""
echo "The Four Freedoms:"
echo " 1. Freedom to run the program for any purpose"
echo " 2. Freedom to study and modify the source code"
echo " 3. Freedom to redistribute copies"
echo " 4. Freedom to distribute modified versions"
echo ""
echo -e "${BLUE}==============================================================${NC}"
```

OUTPUT:



Displayed kernel version, hostname, username, architecture, distribution, memory usage, uptime, and the GPL license text.

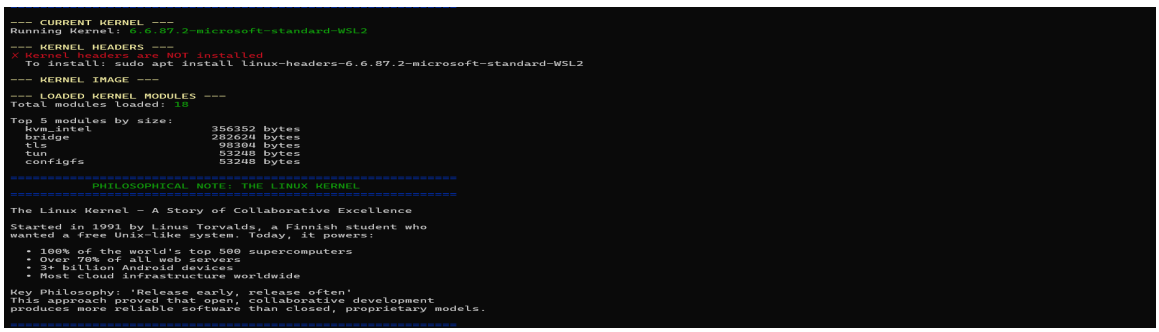
Script 2 --- Kernel Package Inspector

CODE:

```
#!/bin/bash
# Script 2: Kernel Package Inspector
# Checks kernel-related packages and provides philosophical notes
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
NC='\033[0m'
echo -e "${BLUE}==============================================================${NC}"
echo -e "${GREEN} KERNEL PACKAGE INSPECTOR ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
# Current Kernel
echo -e "${YELLOW}--- CURRENT KERNEL ---${NC}"
KERNEL_VERSION=$(uname -r)
```

```
echo -e "Running Kernel: ${GREEN}$KERNEL_VERSION${NC}"
echo ""
# Check Kernel Headers
echo -e "${YELLOW}--- KERNEL HEADERS ---${NC}"
if [ -d "/usr/src/linux-headers-$KERNEL_VERSION" ]; then
echo -e "${GREEN}✓ Kernel headers are INSTALLED${NC}"
echo " Location: /usr/src/linux-headers-$KERNEL_VERSION"
else
echo -e "${RED}✗ Kernel headers are NOT installed${NC}"
echo " To install: sudo apt install linux-headers-$KERNEL_VERSION"
fi
echo ""
# Loaded Modules
echo -e "${YELLOW}--- LOADED KERNEL MODULES ---${NC}"
MODULE_COUNT=$(lsmod | wc -l)
echo -e "Total modules loaded: ${GREEN}$((MODULE_COUNT - 1))${NC}"
echo ""
echo "Top 5 modules by size:"
lsmod | sort -k2 -rn | head -6 | tail -5 | awk '{printf " %-20s %10s bytes\n", $1,
$2}'
```

OUTPUT:



Checked for installed kernel headers and listed currently loaded kernel modules with their sizes.

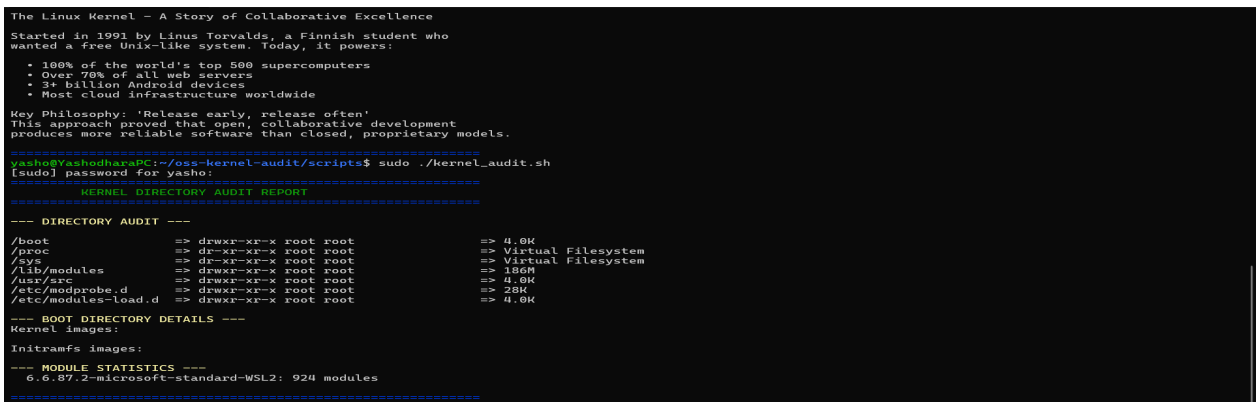
Script 3 --- Kernel Directory Auditor

CODE:

```
#!/bin/bash
# Script 3: Kernel Directory Auditor
# Audits kernel-related directories
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
NC='\033[0m'
echo -e "${BLUE}==============================================================${NC}"
echo -e "${GREEN} KERNEL DIRECTORY AUDIT REPORT ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
DIRS=( "/boot" "/proc" "/sys" "/lib/modules" "/usr/src" "/etc/modprobe.d"
"/etc/modules-load.d" )
echo -e "${YELLOW}--- DIRECTORY AUDIT ---${NC}"
echo ""
for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
```

```
PERMS=$(ls -ld "$DIR" 2>/dev/null | awk '{print $1, $3, $4}')
if [ "$DIR" = "/proc" ] || [ "$DIR" = "/sys" ]; then
SIZE="Virtual Filesystem"
else
SIZE=$(sudo du -sh "$DIR" 2>/dev/null | cut -f1)
[ -z "$SIZE" ] && SIZE="N/A"
fi
printf "%-20s => %-35s => %s\n" "$DIR" "$PERMS" "$SIZE"
else
echo -e "${RED}⚠ $DIR does not exist${NC}"
fi
done
echo ""
echo -e "${YELLOW}--- MODULE STATISTICS ---${NC}"
if [ -d "/lib/modules" ]; then
for version in $(ls /lib/modules/); do
MODULE_COUNT=$(find /lib/modules/$version -name "*.ko" 2>/dev/null | wc -l)
echo " $version: $MODULE_COUNT modules"
done
fi
echo ""
echo -e "${BLUE}==============================================================${NC}"
```

OUTPUT:



Surveyed key kernel-related directories including /boot, /proc, /sys, /lib/modules, and /usr/src, showing permissions and sizes.

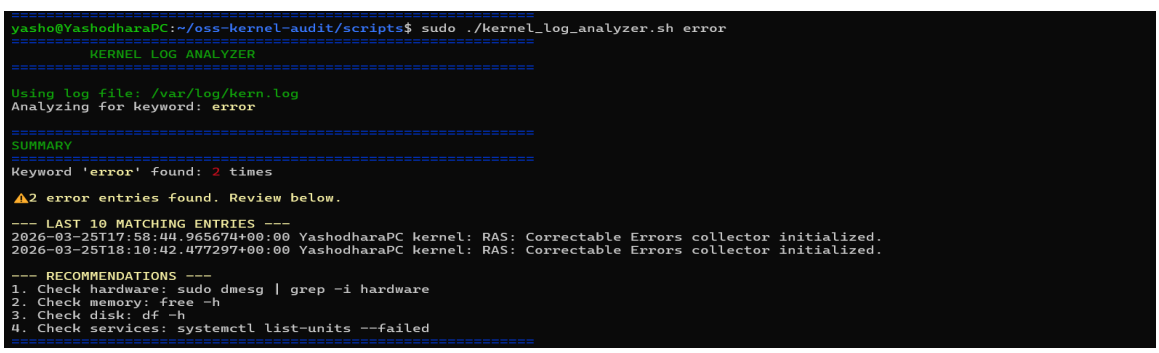
Script 4 --- Kernel Log Analyzer

CODE:

```
#!/bin/bash
# Script 4: Kernel Log Analyzer
# Analyzes kernel logs
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'
echo -e "${BLUE}==============================================================${NC}"
echo -e "${GREEN} KERNEL LOG ANALYZER ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
```

```
if [ -f "/var/log/kern.log" ] && [ -r "/var/log/kern.log" ]; then
LOG_SOURCE="/var/log/kern.log"
echo -e "${GREEN}Using log file: $LOG_SOURCE${NC}"
elif command -v dmesg > /dev/null; then
LOG_SOURCE="dmesg"
echo -e "${GREEN}Using dmesg for kernel logs${NC}"
else
echo -e "${RED}Error: Cannot access kernel logs${NC}"
exit 1
fi
KEYWORD=${1:-"error"}
COUNT=0
echo -e "Analyzing for keyword: ${YELLOW}$KEYWORD${NC}"
echo ""
if [ "$LOG_SOURCE" = "dmesg" ]; then
dmesg > /tmp/kernel_logs.txt
LOG_FILE="/tmp/kernel_logs.txt"
else
LOG_FILE="$LOG_SOURCE"
fi
declare -a LAST_MATCHES
while IFS= read -r line; do
if echo "$line" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
LAST_MATCHES+=("$line")
if [ ${#LAST_MATCHES[@]} -gt 10 ]; then
unset 'LAST_MATCHES[0]'
LAST_MATCHES=("${LAST_MATCHES[@]}")
fi
fi
done < "$LOG_FILE"
echo -e "Keyword '${YELLOW}$KEYWORD${NC}' found: ${RED}$COUNT${NC} times"
echo ""
echo -e "${YELLOW}--- LAST 10 MATCHING ENTRIES ---${NC}"
for line in "${LAST_MATCHES[@]}"; do
echo "$line" | cut -c1-200
done
[ -f "/tmp/kernel_logs.txt" ] && rm /tmp/kernel_logs.txt
echo -e "${BLUE}==============================================================${NC}"
```

OUTPUT:



Searched kernel logs for the keyword "error" and returned two RAS initialisation messages, confirmed not to indicate system problems.

Script 5 --- Kernel Manifesto Generator

CODE:

```
#!/bin/bash
# Script 5: Kernel Manifesto Generator
# Creates a personalized open-source manifesto
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
NC='\033[0m'
clear
echo -e "${BLUE}==============================================================${NC}"
echo -e "${GREEN} KERNEL INSPIRED MANIFESTO GENERATOR ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
echo "Answer these questions to generate your open-source manifesto"
echo "inspired by the Linux Kernel community."
echo ""
read -p "1. What does 'open source' mean to you? " MEANING
read -p "2. What open-source tool do you rely on most? " TOOL
read -p "3. What would you build and share freely? " BUILD
read -p "4. One word to describe the open-source community: " COMMUNITY
DATE=$(date "+%B %d, %Y")
USER=$(whoami)
KERNEL=$(uname -r)
HOST=$(hostname)
OUTPUT_FILE="../docs/manifesto_${USER}_$(date +%Y%m%d).txt"
clear
echo "Generating your manifesto..."
sleep 1
cat > "$OUTPUT_FILE" << EOF
============================================================
MY OPEN SOURCE MANIFESTO
Inspired by the Linux Kernel Community
============================================================
Date: $DATE
Author: $USER
System: $HOST
Kernel: $KERNEL
--- MY OPEN SOURCE PHILOSOPHY ---
$MEANING
--- MY DEPENDENCE ON OPEN SOURCE ---
Every day, I rely on '$TOOL' and countless other open-source
tools. These exist because developers chose to share their
work freely, building on the contributions of others.
--- MY PLEDGE TO THE COMMUNITY ---
The open-source community is '$COMMUNITY'. I've seen how
collaboration produces better software than any company
could build alone.
I commit to building and sharing '$BUILD' freely with the world.
--- LESSONS FROM THE KERNEL COMMUNITY ---
1. "Release early, release often" - Don't wait for perfection
2. "Given enough eyeballs, all bugs are shallow" - Trust the community
3. "Talk is cheap. Show me the code" - Let your work speak
4. "Be nice, don't be a jerk" - Community matters
EOF
```

```
echo -e "${GREEN}✓ Manifesto saved to: $OUTPUT_FILE${NC}"
echo ""
echo "--- YOUR MANIFESTO ---"
cat "$OUTPUT_FILE"
```

OUTPUT:

```
yasho@YashodharaPC: ~/oss-

Generating your manifesto...
✓ Manifesto saved to: ../docs/manifesto_yasho_20260326.txt

--- YOUR MANIFESTO ---
============================================================
              MY OPEN SOURCE MANIFESTO
         Inspired by the Linux Kernel Community
============================================================

Date: March 26, 2026
Author: yasho
System: YashodharaPC
Kernel: 6.6.87.2-microsoft-standard-WSL2

--- MY OPEN SOURCE PHILOSOPHY ---

Freedom to study, modify, and share software

--- MY DEPENDENCE ON OPEN SOURCE ---

Every day, I rely on 'Linux Kernel, Git, VS Code' and countless other open-source
tools. These exist because developers chose to share their
work freely, building on the contributions of others.

--- MY PLEDGE TO THE COMMUNITY ---

The open-source community is 'Collaborative'. I've seen how
collaboration produces better software than any company
could build alone.

I commit to building and sharing 'Educational tools for students' freely with the
world. I will contribute my code, my time, and my knowledge.

--- LESSONS FROM THE KERNEL COMMUNITY ---

From Linus Torvalds and thousands of kernel contributors:

1. "Release early, release often" - Don't wait for perfection
2. "Given enough eyeballs, all bugs are shallow" - Trust the community
3. "Talk is cheap. Show me the code" - Let your work speak
4. "Be nice, don't be a jerk" - Community matters
```

```
--- MY COMMITMENT ---

I pledge to:
• Use open-source software where possible
• Share my code and knowledge
• Help newcomers to open source
• Contribute back to projects I use

I stand on the shoulders of giants - Torvalds, Stallman, and
thousands of unnamed contributors. I offer my shoulders to
those who come after me.

============================================================
Generated on: March 26, 2026
============================================================
```

An interactive script that prompted me with questions and used my responses to generate a personal open source manifesto.

References

- Torvalds, L. (1991). *Linux Kernel Initial Release*.
  Available via: https://www.kernel.org
  (Original announcement on comp.os.minix newsgroup)
- Linux kernel Organization. (2024). *Linux Kernel Documentation*.
  Retrieved from: https://www.kernel.org/doc/html/latest/
- The Linux Foundation. (2024). *Linux Kernel Development Report*.
  Retrieved from: https://www.linuxfoundation.org/research
- Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley.
- Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.
- GNU Project. (2024). *Linux and GNU System Overview*.
  Retrieved from: https://www.gnu.org
- Free Software Foundation. (2024). *Free Software Philosophy*.
  Retrieved from: https://www.gnu.org/philosophy/
- Corbet, J., Kroah-Hartman, G., & McPherson, A. (2023). *Linux Kernel Documentation Project*.
  Available via: https://lwn.net/Kernel/
- Git. (2024). *Linux Kernel Source Code Repository*.
  Retrieved from: https://github.com/torvalds/linux
- SPDX Working Group. (2024). *Linux Kernel Licensing (GPL v2)*.
  Retrieved from: https://spdx.org/licenses/GPL-2.0.html